How-To Guide - Group 37

Aryan Saadati | Trent Battisti | Mani Samet

Data Cleaning and Processing

The data in this project was divided into two categories: human gameplay and Stockfish model gameplay. The Stockfish model gameplay data was generated by us by simulating chess games where the model played against itself. We collected the data from those games and stored them in PGN format using the Python Chess library, which is the format most commonly used for chess data storage. The human gameplay data used in this project was also downloaded in PGN format, which is a format designed for storing chess game data. The data came in a clean and interpretable format, leaving us little to no work to do for cleaning the data.

As for pre-processing the data, we used two main approaches. One approach utilized Pandas, or for larger files Spark, Library and the Python chess library to convert the files to CSV format. These files were then processed and filtered based on the different model requirements in CSV format and were converted back to PGN for training use. One use case of this was filtering games based on ELO. The other approach filtered data at training time, discarding certain inputs as per model requirements, such as filtering the early moves in the game for transfer learning.

The input training data is represented by an 8x8x12 tensor. Each of the twelve 8x8 layers represents a chessboard whose entries track the position of each type of chess piece. These twelve layers represent the six white pieces and the six black pieces (pawn, knight, bishop, rook, queen, and king for each color). Each layer tracks the positions of a single piece type on the board, with a value of 1 marking the presence of that piece on a given square and 0 indicating its absence. This representation uniquely represents each state. It is also sparse; the number of non-zero entries does not exceed 32, which is roughly 4% of the entries. Also, as the game goes on, the number of pieces on the board can only go down, resulting in an even more sparse tensor representation. On top of that, the entries only consist of 1s and 0s. Altogether, this tensor representation is simple and ideal for techniques that are commonly used in the numerical methods of training neural network models.

The labels for the training data were collected by looping through each game and at each state, collecting the subsequent move done as a label for that state.

Model Configuration and Tuning

For configuring our model two initial convolutional layers, the first one with 64 filters of size 3x3 and a second with 128 filters of size 3x3. These layers capture the spatial feature of the board, specifically in regards to the 6 unique pieces for each side. We limited our model to only two convolutional layers since adding a third larger convolutional layer would capture irrelevant spatial relations, decreasing the performance of our model. These spatial features were then condensed by our flattened layer so they could be analyzed by our following dense layers. Our first dense layer has 256 neurons to learn these spatial relationships extracted from the convolutional layers. We chose 256 since this kept our model to a manageable size,

so we would keep our training time to just under an hour while maximizing our performance. Then our final dense layer and output layer took these learned spatial relationships and created a probability distribution across all of the possible moves in our dataset. As our model trained, it would be able to learn and recognize the spatial relationships between the pieces for various board states.

Overall, our convolutional layers used ReLu as the activation function. This was chosen after testing out multiple activation functions like GeLu and Swish. Both of which performed worse than ReLu. ReLu was also used for the dense layers for similar performance reasons. Except for the output layer, which used Softmax to standardize the results and create the probability distribution.

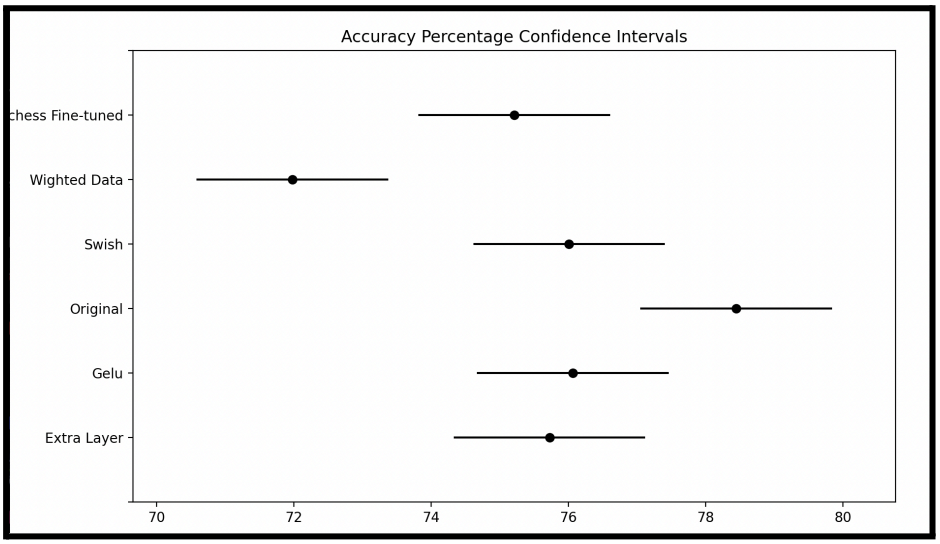

Figure 1: *Accuracy performance for various model configurations using different activation functions, an added convolution layer and weighted data.*

For configuring our transfer learning model, we initially froze the first three layers, the two convolutional layers and the flatten layer. This means that while training, these layers wouldn't be updated. Using these as our frozen base, we added three dense layers with two dropout layers in between them. The first dense layer had 2048 neurons. And then a dropout layer of 50%. This was added to counteract the bias that was inherently present in our dataset. Since there are multiple repeated opening and midgame moves, those moves would become over learned and overrepresented in the resulting probability distribution. Ruining our models' late-game performance. After that, we have another dense layer with 1024 neurons, followed by a dropout layer of 40%. And then followed by the final dense layer for our output, which again generates a probability distribution over all of the possible moves seen in the dataset.

For fitting our model, we set a maximum number of 50 epochs with a validation split of 10% and a batch size of 64. We chose 50 epochs since our Adam optimizer has a default learning rate of 0.001, which we found to be a good fit for our initial model architecture. Similarly, we chose a 10% validation split since we were only training with 10k games. If we increased this split, we would likely run into validation issues, primarily due to the number of possible late-game scenarios present in them. We chose a batch size of 64 primarily due to our hardware

constraints, since larger batch sizes would increase our memory demands and may not generalize as well. For our transfer learning models, we also implemented callbacks. Specifically, a callback that tracked the validation accuracy and would stop training if the validation accuracy plateaued or started decreasing over the course of 10 epochs. This implementation was made after monitoring the training process and seeing our validation accuracy begin to decrease by around the 30th epoch for most models. As well, the learning rate for our transfer learning models was decreased to 0.0001 for more stability while training.

Model Evaluation

In order to validate our models, we first used 10% hold-out validation and performed training on 90% of both Simulated data (Stockfish self-play) and human-played matches (used for a transfer learned model).

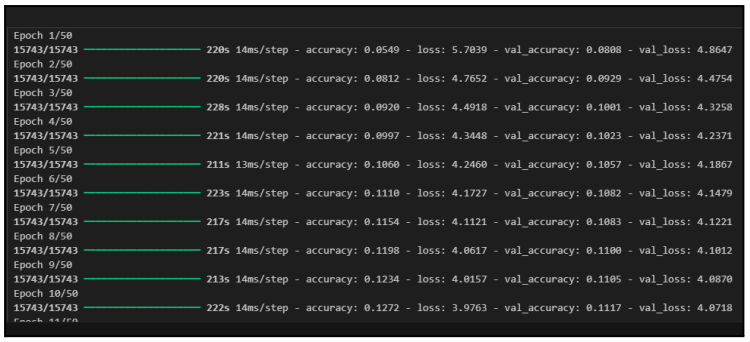

Figure 2: *Illustration of validation and Loss Function in training*

Additionally, our second and third validation methods were based on two chess metrics, average centipawn loss and accuracy percentage.

The centipawn scoring was measured using the Python Chess library, and it was measured before and after each move to calculate the loss from that move. It was then averaged across all the moves throughout the game to evaluate the average centipawn loss.

The accuracy percentage is a method that is adopted from the Lichess Accuracy Metric. In detail, for every move played by our model, Stockfish is queried before and after the move. Both are converted to Win% using the logistic formula as shown here:

$$Win\% = 50 + 50 * (2 / (1 + \exp(-0.00368208 * centipawns)) - 1)$$

Subsequently, the change in win% is plugged into the Lichess Accuracy Percentage per move formula as shown here:

$$Accuracy\% = 103.1668 * \exp(-0.04354 * (winPercentBefore - winPercentAfter)) - 3.1669$$

These per-move accuracies are aggregated by:

1) Dividing the game into sliding windows. The window size is determined by the length of the game.
2) Computing the volatility of each window by calculating its standard deviation. In this case, we use the standard deviation of the Win% of each move in the window.
3) Computing the volatility-weighted mean of the move accuracies using the volatility as weights for each move.
4) Computing the harmonic mean of the move accuracies.
5) Taking the average of the volatility weighted mean and the harmonic mean to yield the final game accuracy.

We tested our models against Stockfish for 100 games, calculated each metric per game and averaged them to produce a comparative evaluation for our models.

Error Analysis

As mentioned above, we utilized our hold-out validation method in combination with the categorical cross-entropy Loss function. This Loss function works best for loss tracking when dealing with our one-hot vector encodings, as well as our multi-class classification requirement. Consequently, we needed a loss function that could compare a probability distribution output (Softmax) to the correctly labelled move.

Gameplay Visualization

The gameplay was visualized through the ipywidgets and Python chess library. We set our models against the Stockfish model and automated their gameplay. The following video is an example of one of our models playing the Stockfish model at 2000 ELO.

CMPT 310, Proof of accomplishment 3

Training Pipeline Refinement

To compile our model, we used the Adam optimizer with categorical cross-entropy for the loss function since we used one-hot encoding for our features. Additionally, although not included in our project, we experimented with integer encoding, ultimately using the sparse categorical cross-entropy loss function. This further improves memory usage, as there is no need to expand labels into giant one-hot vectors. This change would allow us to significantly increase the size of our training data for our model and ultimately improve its performance. As well, we implemented callbacks for our transfer learning models since during our initial training, we saw that their validation accuracy would plateau and begin to decrease by the 30th epoch. This allowed us to stop our model and revert back to the best weights historically when training. Moreover, it saves on training time since our model doesn't have to train through 50 epochs before saving its learning weights.

Sources

Lichess Accuracy Metric: https://lichess.org/page/accuracy
Inspired GitHub Repository: https://github.com/Skripkon/chess-engine.git